

EEE3095/6S Final Exam

November 2023

Duration: 3 hours

Total: 150 marks

Empl ID: _____

Instructions:

- This is a closed-book exam.
- There are **two** sections (Parts A and B) to this exam.
- You must use a pen to complete the exam – pencil will only be marked for drawings. Values indicated on drawings must be written in pen.
- Please show all your working clearly – your method and approach matter.
- Keep all your answers within the allocated blocks; anything outside of these blocks may not be marked.
- Two reference sheets appear at the end of this paper.

Question	Available Marks	Grade
1	10	
2	15	
3	20	
4	15	
5	15	
6	12	
7	16	
8	16	
9	15	
10	16	
Total:	150	

Part AEmpl ID:

--	--	--	--	--	--	--

Question:	1	2	3	4	5	Total
Points:	10	15	20	15	15	75
Score:						

Question 1: Multiple Choice [10 marks]

- a. If we have a real-time system that can survive task failures as long as the missed deadlines are infrequent, what kind of system is this said to be? (2)
- ☐ Real real-time
☐ Firm real-time
☐ Soft real-time
☐ Hard real-time
- b. According to the Brisbane definition, an embedded system is described as: (2)
- ☐ an information processing system embedded into a larger product
☐ a system that integrates computation with physical dynamics
☐ a task-specific computer, which is built into a larger system for the purpose of controlling and monitoring the larger system
☐ a computer system in which a mechanism is controlled or monitored by computer-based algorithms
- c. In a 4-bit system, what would be the base-10 value of 0b1001 in a signed representation? (2)
- ☐ 7
☐ -9
☐ 9
☐ -7
- d. A CISC processor is restricted to only working on operands that are stored in registers. True or false? (2)
- ☐ True ☐ False
- e. RS-232 is an asynchronous serial signaling standard that uses the voltage range (+3 V to +15 V) as a logic 1 and (-3 V to -15 V) as a logic 0. True or false? (2)
- ☐ True ☐ False

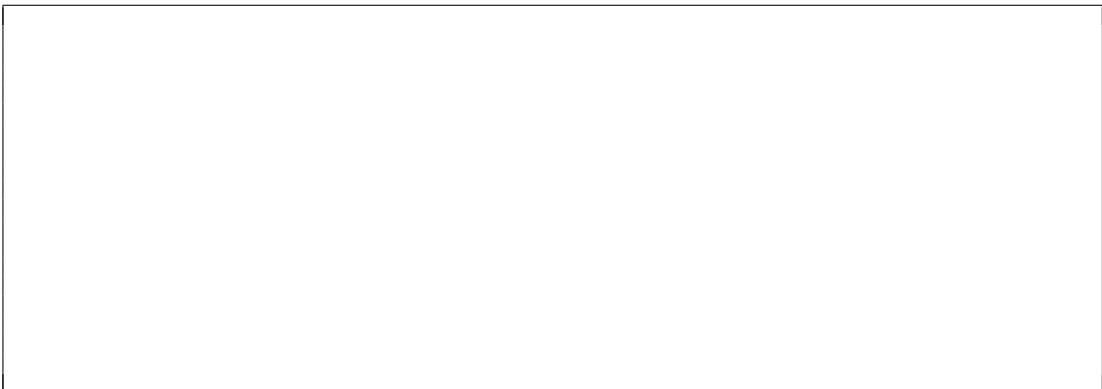
Question 2: ARM Processing [15 marks]

- a. Describe the term "development environment" in the context of both PC and embedded systems development, and identify at least two differences between them. (4)



- b. If we have access to ARM, AVR and PIC microprocessors, which would be most suited to each of the following devices? Give a brief reason for each answer. (4)

1. Apple iPhone
2. ABS braking system
3. Weather sensing device
4. Burglar alarm





c. C is a very popular, powerful and flexible programming language that is often called a "portable Assembly language". Answer the following questions:

(i) In what way is C seen as "portable Assembly" compared to, for example, Python or Java?

(2)



- (ii) Why is C generally preferred over C++ for embedded systems development? Provide two reasons. (2)

- (iii) What would happen if the following command was successfully run in a terminal? (3)

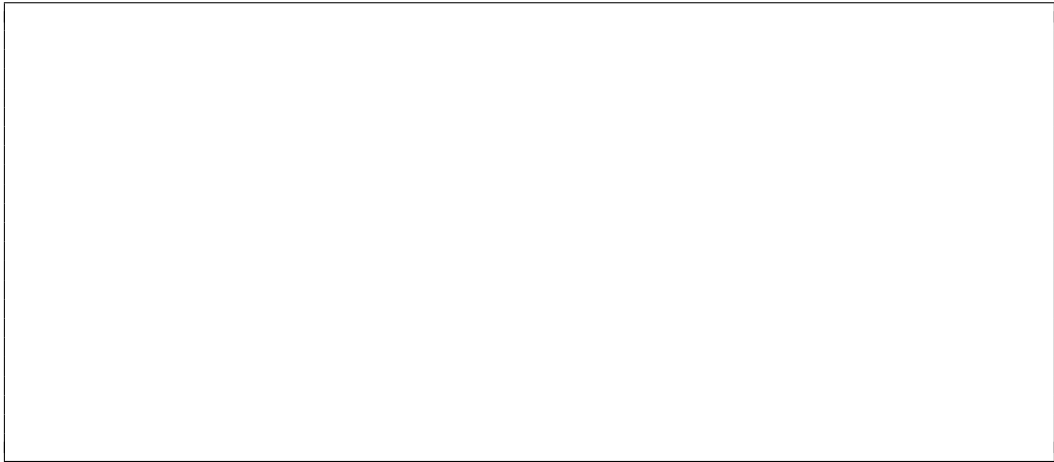
```
gcc -O1 main.c -o myprog
```

Question 3: Embedded Communications [20 marks]

a. RS-232 is known to have some limitations compared to (the newer) RS-485.

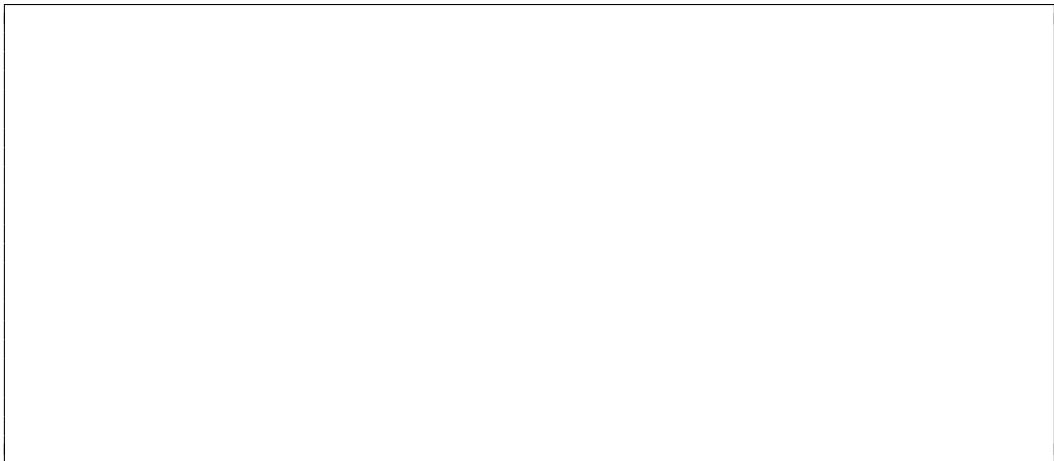
(i) List two limitations associated with RS-232.

(2)



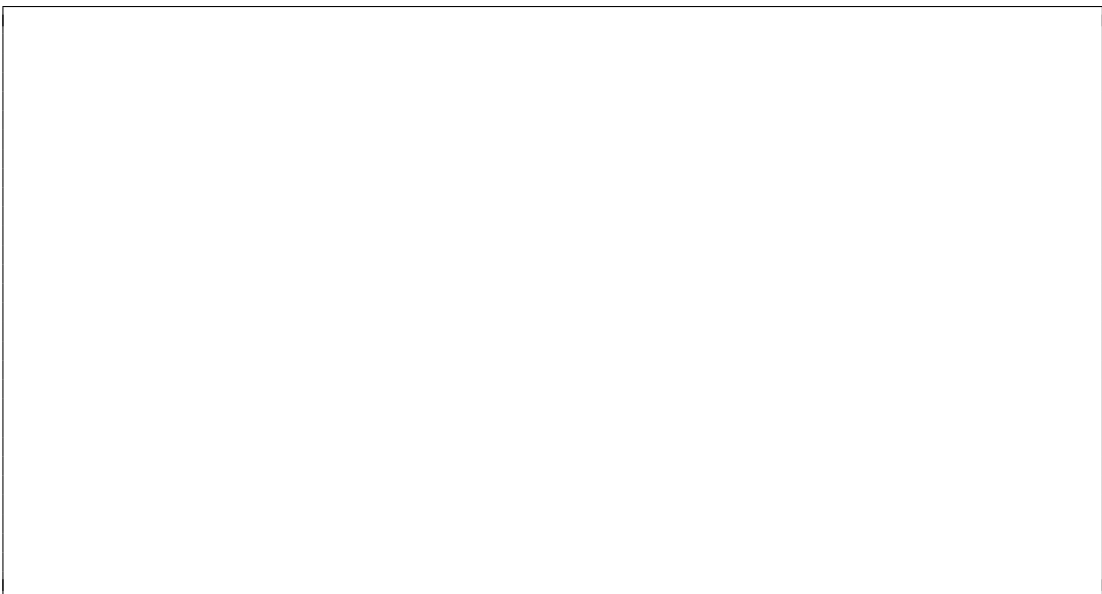
(ii) List two advantages associated with RS-485.

(2)



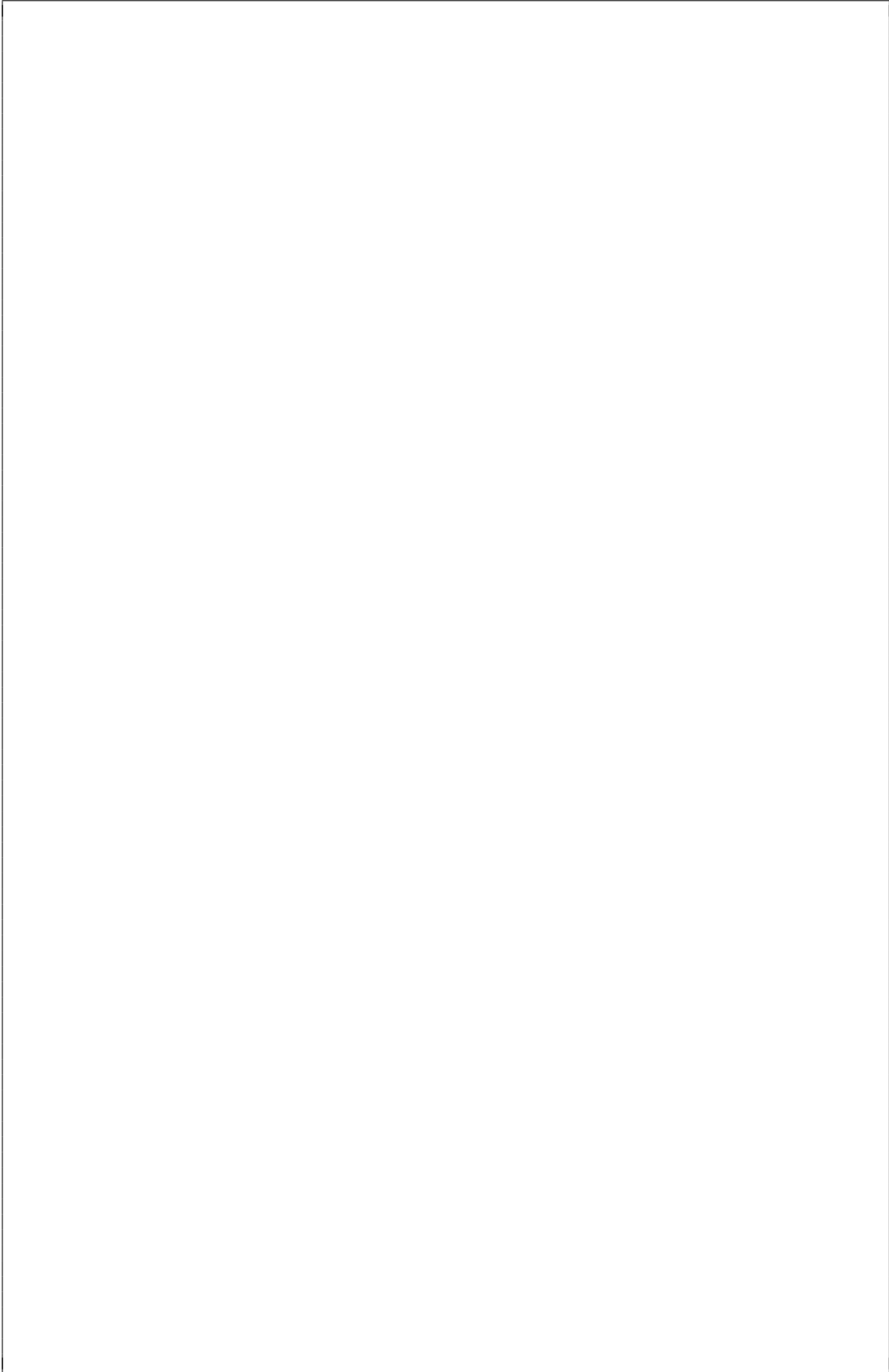
b. If we have an 8-bit data word 0x6A that needs to be transmitted with a parity bit appended, what would be the final 9-bit data word that is transmitted for both odd and even parity?

(2)



- c. Draw a labelled diagram of the physical I2C interface for one master and two slave devices, including all signal lines and slave addresses, and then briefly explain each aspect of your illustration.

(8)



- d. The following sub-questions relate to the SPI communication protocol. (4)
- (i) The four SPI Modes are dependent on the Clock Phase (CPHA) and Clock Polarity (CPOL). Draw a table to illustrate how these two quantities vary based on the configured SPI Mode, and include a very brief description of each of the two terms.

SPI Mode	CPHA	CPOL	Description
Mode 0	Leading edge	High	Master Out Slave In (MOSI)
Mode 1	Trailing edge	High	Master In Slave Out (MISO)
Mode 2	Leading edge	Low	Master In Slave Out (MISO)
Mode 3	Trailing edge	Low	Master Out Slave In (MOSI)

- (ii) SPI allows only one master device to be connected, but multiple slave devices can be used. Briefly explain how we could connect up multiple *independent* slaves using the same SPI bus. (2)

Question 4: ARM Assembly [15 marks]

The following questions relate to ARM Assembly code being run on a 32-bit ARM processor.

- a. You are given the ARM assembly code below. Answer the following questions.

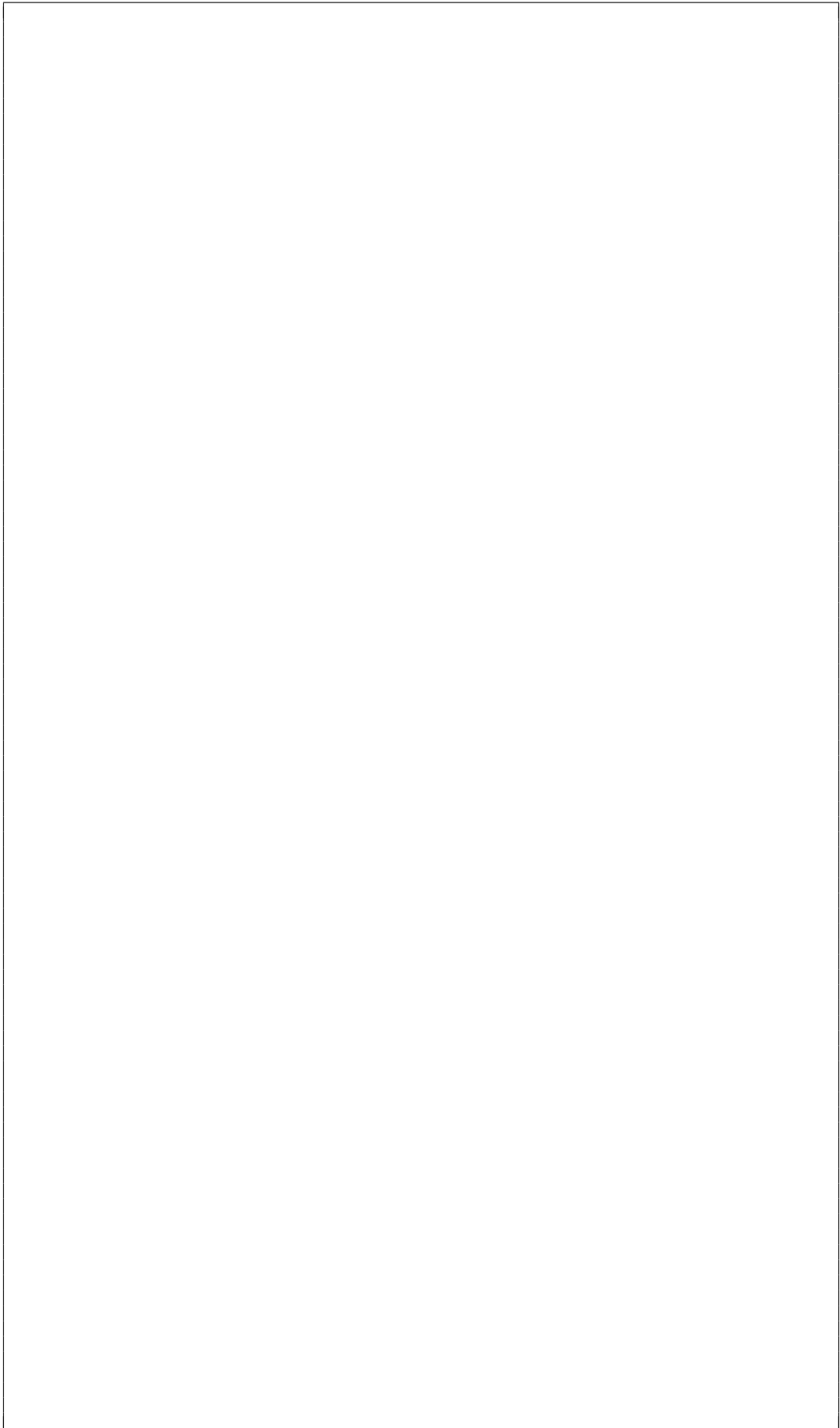
```
myfunc: @ void myfunc (int input); modifies a global variable myglob
        cmp r0, #0
        rsbmi r0, r0, #0
        ldr r1, =myglob @ r1 = address of myglob
loop:
        ldr r2, [r1] @ get value of myglob
        sub r2, r2, r0
        sub r2, r2, r0
        sub r2, r2, r0
        sub r2, r2, r0
        cmp r2, #0
        bgt loop
        str r2, [r1]
        ldr r2, [r1]
        bx lr @ return from function myfunc
```

- (i) Briefly explain what the `rsbmi` instruction would do in this program.

(2)

- (ii) Explain how you could optimise this program; you could do this by rewriting the optimised version of the full code and commenting on the reason for the changes, or by simply explaining each aspect of what you would change in the code (and why).

(6)



- b. You are asked by a client to implement the following mathematical operation in ARM Assembly:

(7)

$$r0 = ((r1 * (r2 + 4))/8) - 4$$

If the result is negative, the value should also be squared. Write an appropriate piece of Assembly code that will meet the client's requirements, and show all reasoning using appropriate Assembly comments in your code. At the end of your code, include a `bx lr` instruction to return to a calling function.

Question 5: Embedded Design [15 marks]

- a. A stack works by taking items and piling them one on top of the other. In computing, these items would be pieces of data and we would typically have two primary functions to interact with the stack, namely:

(6)

1. **push(x)**: adds the object/value **x** to the top of the stack
2. **x = pop()**: removes the topmost object/value from the stack and return it in **x**

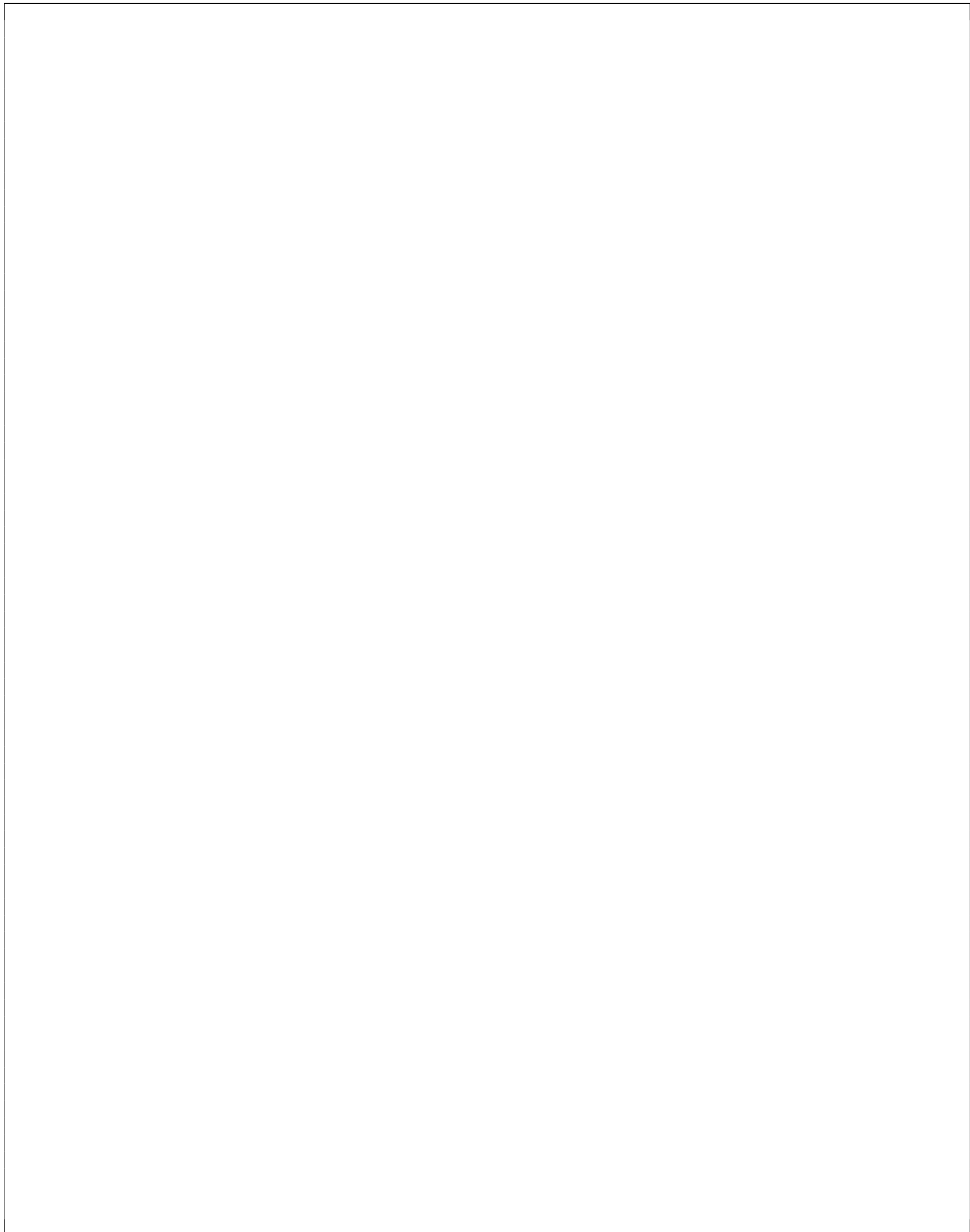
Write code to show how you would implement this in the C programming language. Use the `unsigned int` data type for your stack elements and a maximum stack size of 1000.

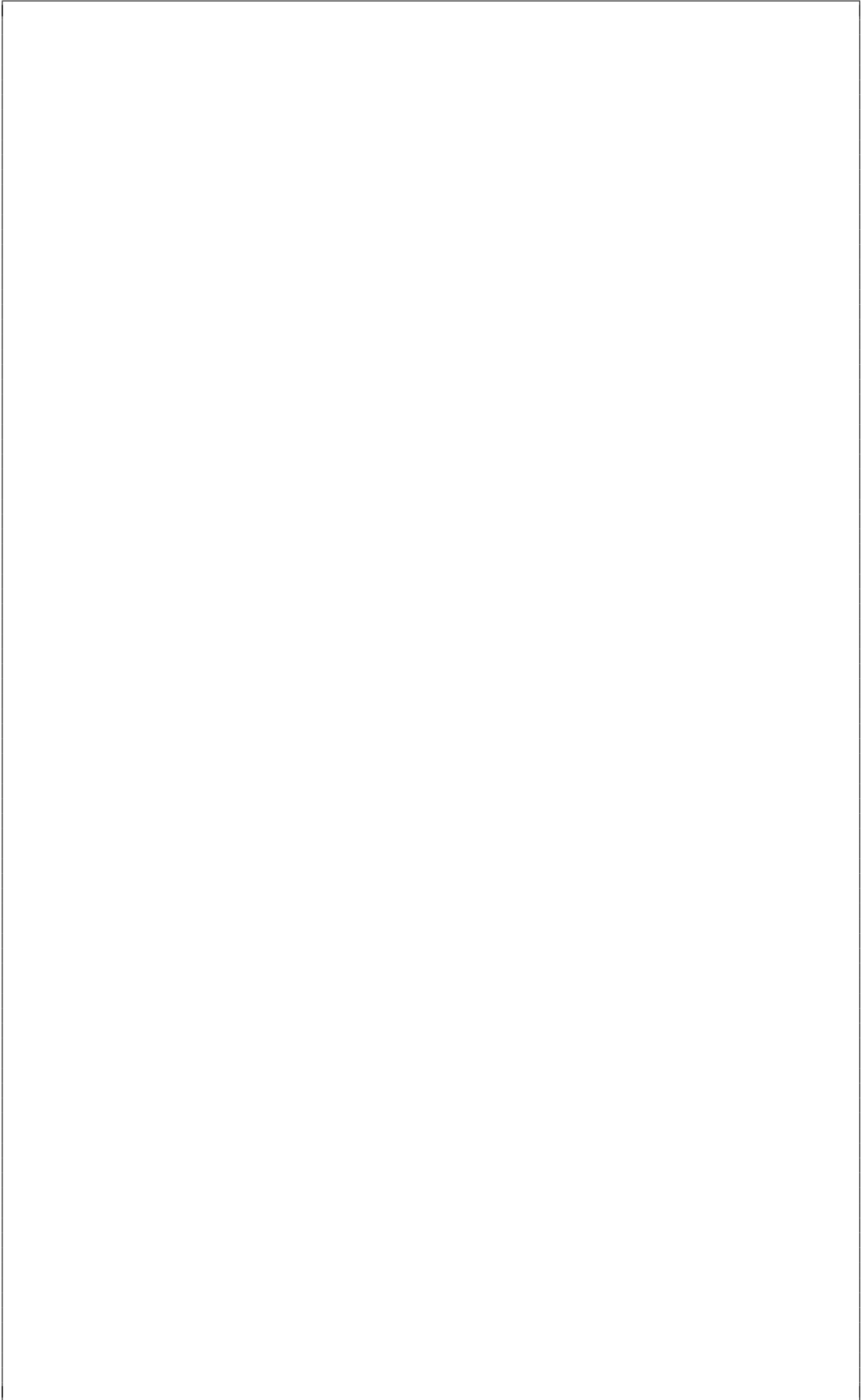
b. You are asked to develop embedded software (for your STM board) that achieves the following:

(9)

- Sample the value from potentiometer POT0; assume it has been configured in 12-bit mode
- Output the current value being sampled (from POT0) to the LCD screen on your STM board
- If the ADC reading is at or above the halfway point (i.e., half the maximum ADC sample value), LED0 should be turned on; if the current reading is below the halfway point, turn LED0 off
- Implement a button interrupt for push-button PA0 such that, when pressed, the interrupt handler should toggle the state of LED7; you may ignore debouncing

Draw a labelled flowchart showing how you could go about implementing all of the above, showing your logic clearly.





Part BEmpl ID:

--	--	--	--	--	--	--

Question:	6	7	8	9	10	Total
Points:	12	16	16	15	16	75
Score:						

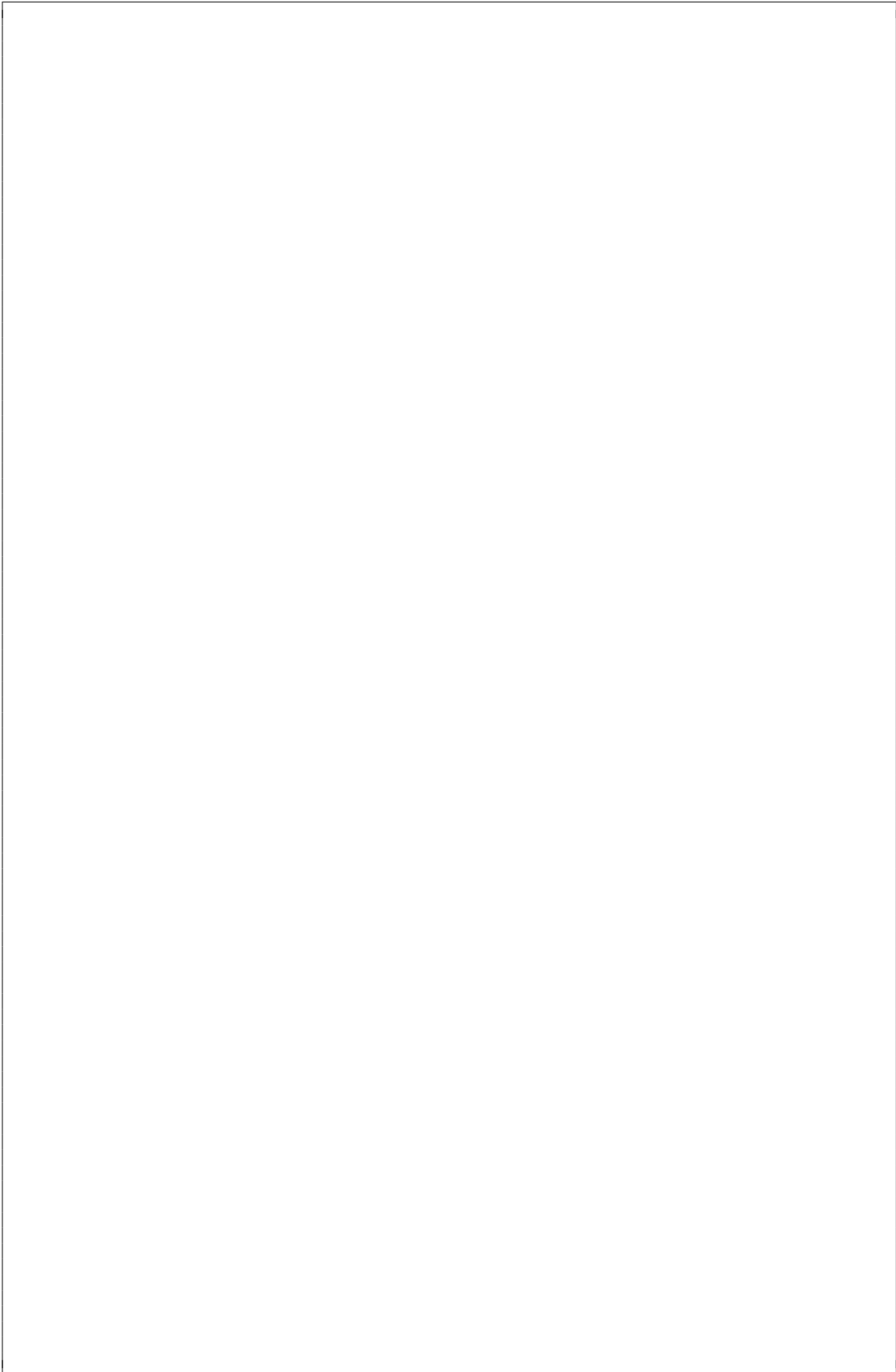
Question 6: Multiple Choice [12 marks]

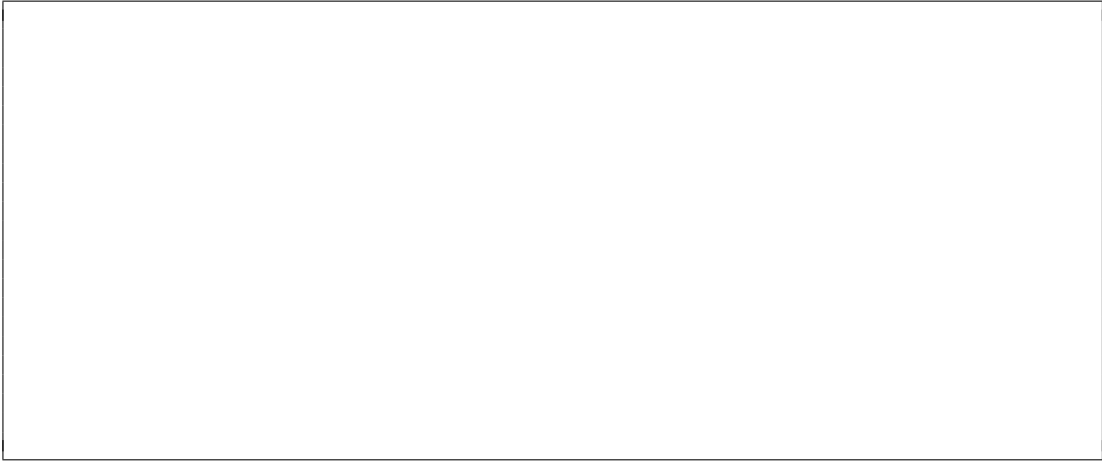
- a. Among these options, which one does **not** constitute a part of the 16-bit register frame structure? (2)
- ☐ Function Unit (FU)
 - ☐ Opcode
 - ☐ Destination register (DR)
 - ☐ Source register A (SA)
- b. For a typical telephone line with a signal-to-noise ratio of 40 dB and an audio bandwidth of 3 kHz, what would be the maximum data rate? (4)
- ☐ 29,901 bps
 - ☐ 30,002 bps
 - ☐ 39,870 bps
 - ☐ 19,910 bps
- c. "A band-limited analog signal that has been sampled can be perfectly reconstructed from an infinite sequence of samples if the sampling rate f_s exceeds $2f_{max}$ samples per second, where f_{max} is the highest frequency in the original signal". Which theory is this? (2)
- ☐ Shannon's theory
 - ☐ Nyquist's theory
 - ☐ Thevenin's theory
 - ☐ Berlo's theory
- d. Spatial computation is typical of programmers. True or false? (2)
- ☐ True
 - ☐ False
- e. A simple 2-select line multiplexer includes four AND gates and one OR gate. True or false? (2)
- ☐ True
 - ☐ False

Question 7: ADCs and DACs [16 marks]

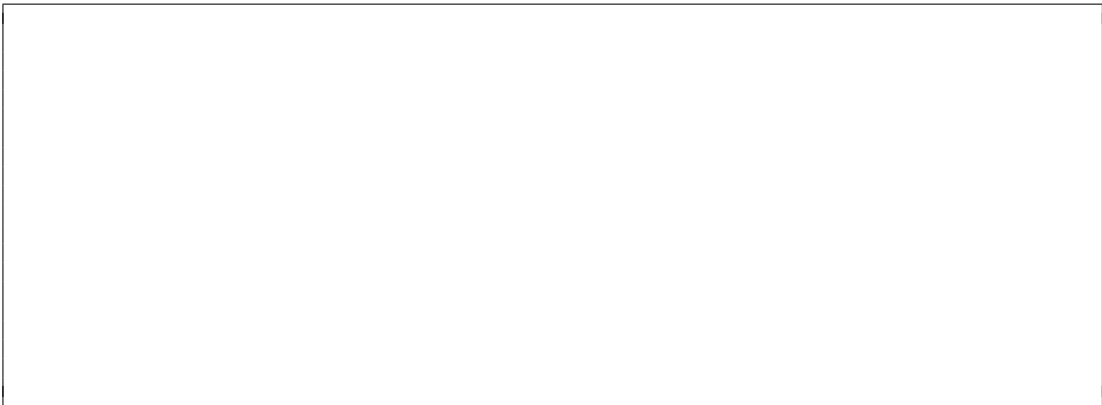
- a. Multistage and pipelined flash Analog-to-Digital Converters (ADCs) have some similarities in their operation. How would you differentiate the two? Use diagrams in your explanation.

(10)

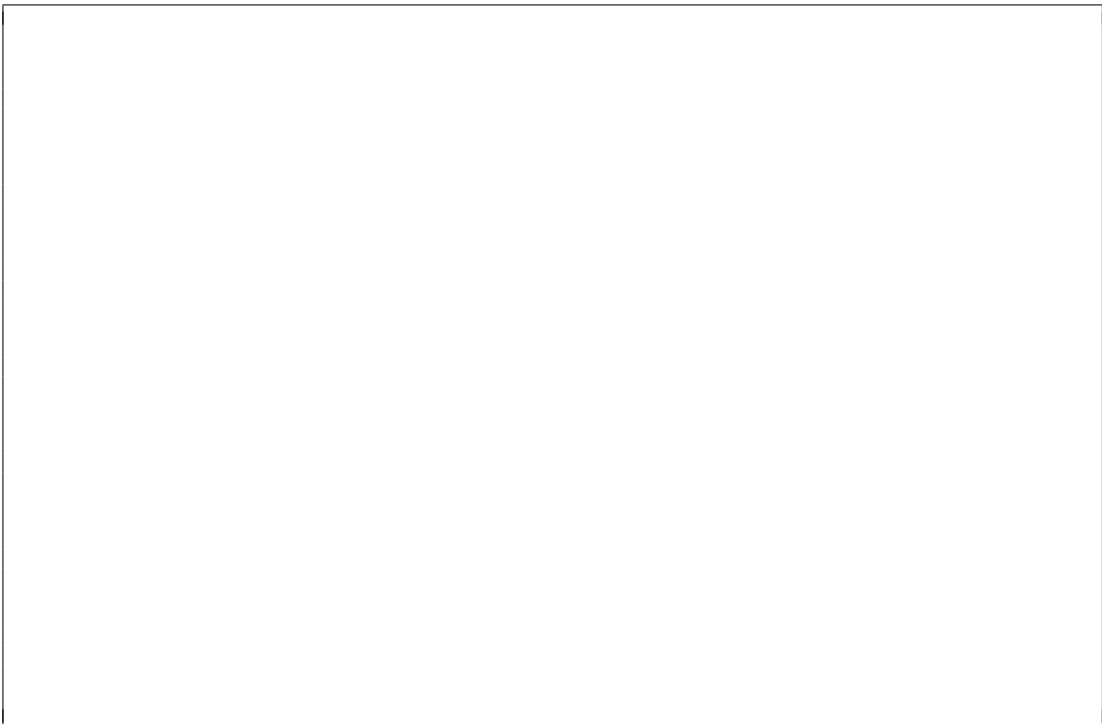




- b. What would be the quantisation step size (in Volts) of a 16-bit ADC with a voltage range of 0 to 4.5 V? (2)



- c. For a 4-bit R2R ladder DAC: if the voltage output is 3.25 V, determine the equivalent digital input signal in binary. Assume a reference voltage of 5 V. (4)



Question 8: Finite State Machines [16 marks]

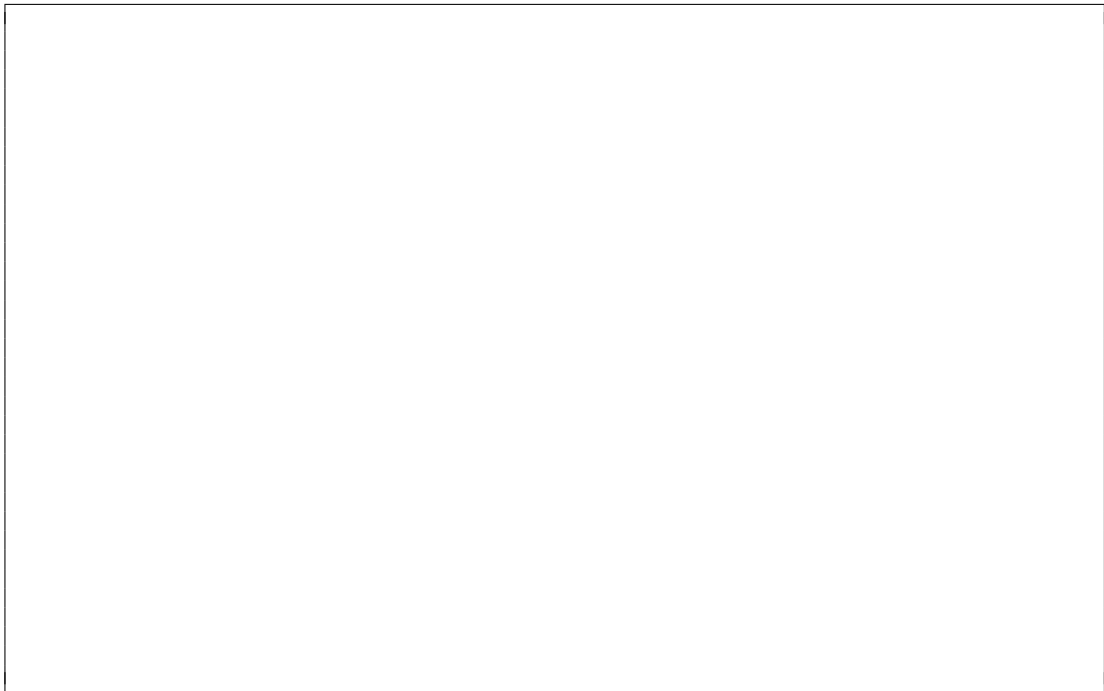
Imagine you own a tech design startup and a client has asked you to design an automatic tea/coffee vending machine based on an FSM model. The customer has given you the following system requirements that you must adhere to:

1. The tea/coffee is initiated by the user inserting a R5 coin.
2. After inserting the coin, the user can either select "Coffee" or "Tea", or press "Cancel" to revoke the order and take back the coin. The order is automatically cancelled (and the coin is pushed back) if a 10-second timer expires.
3. If the user selects coffee or tea, coffee or tea should be correctly dispensed.

Answer the following questions.

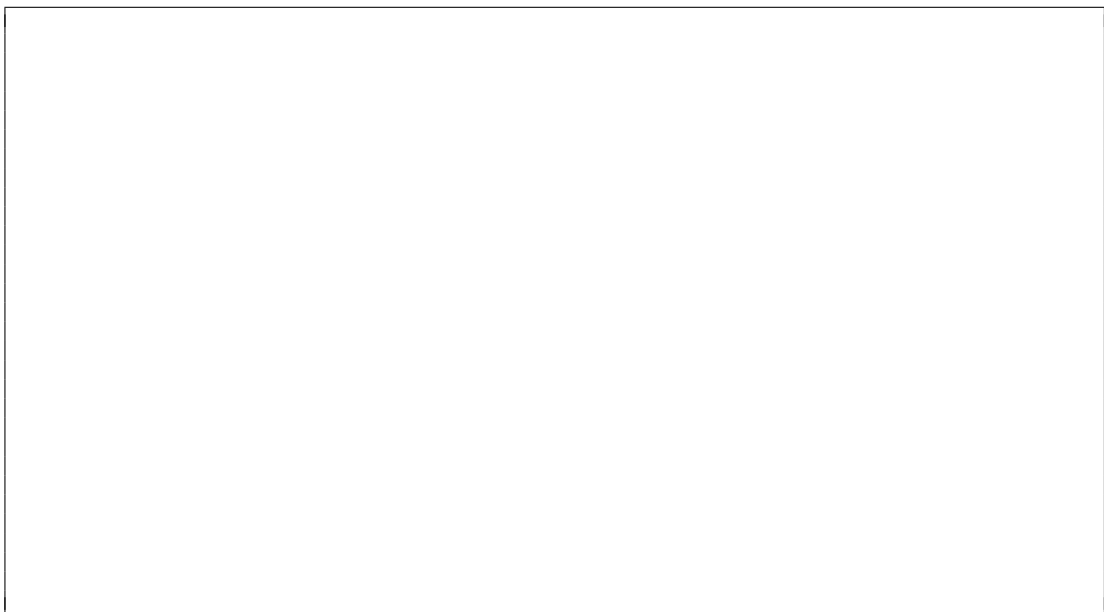
- a. What are the states of the design?

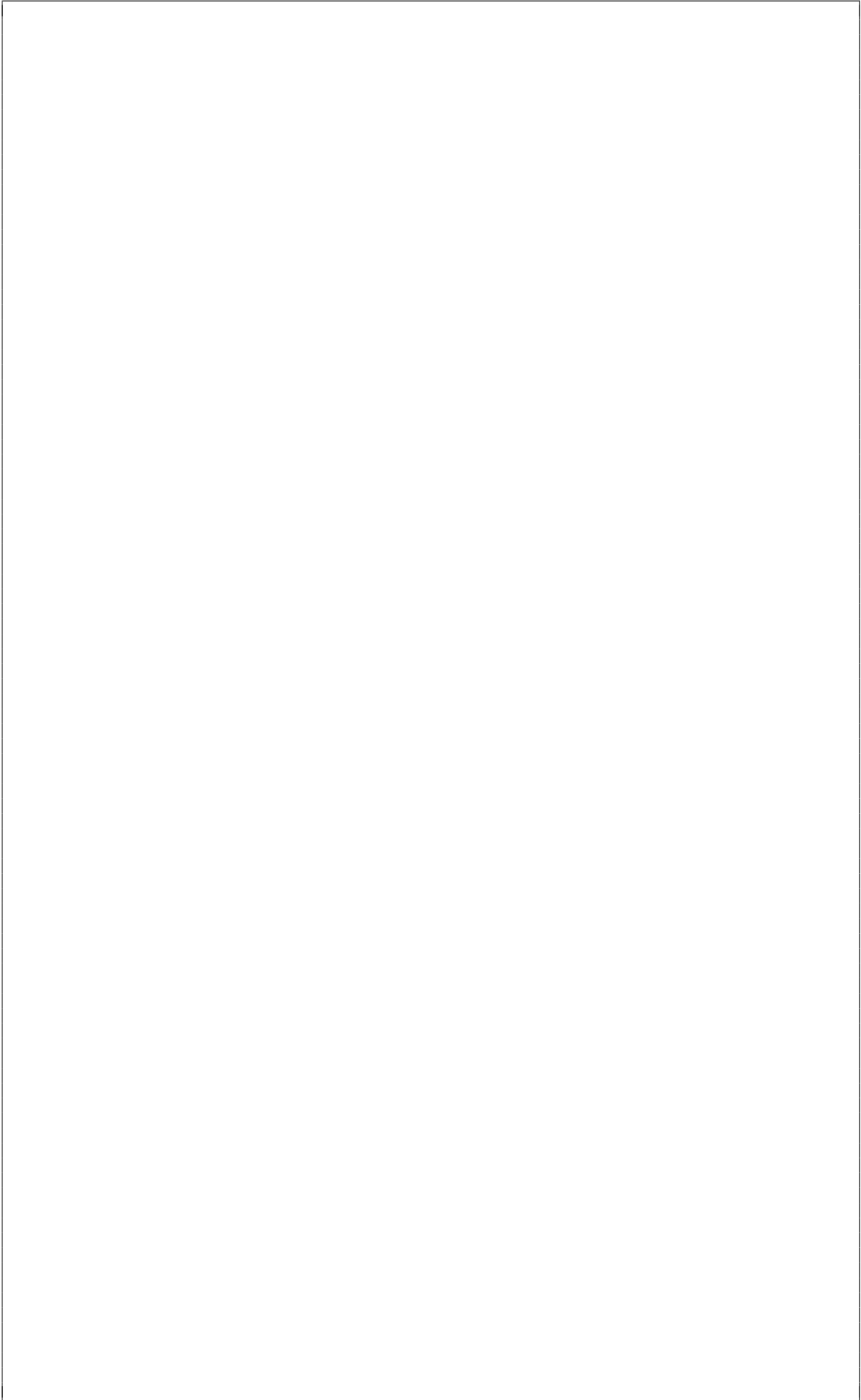
(4)



- b. Draw the finite state model diagram showing all states and actions.

(12)



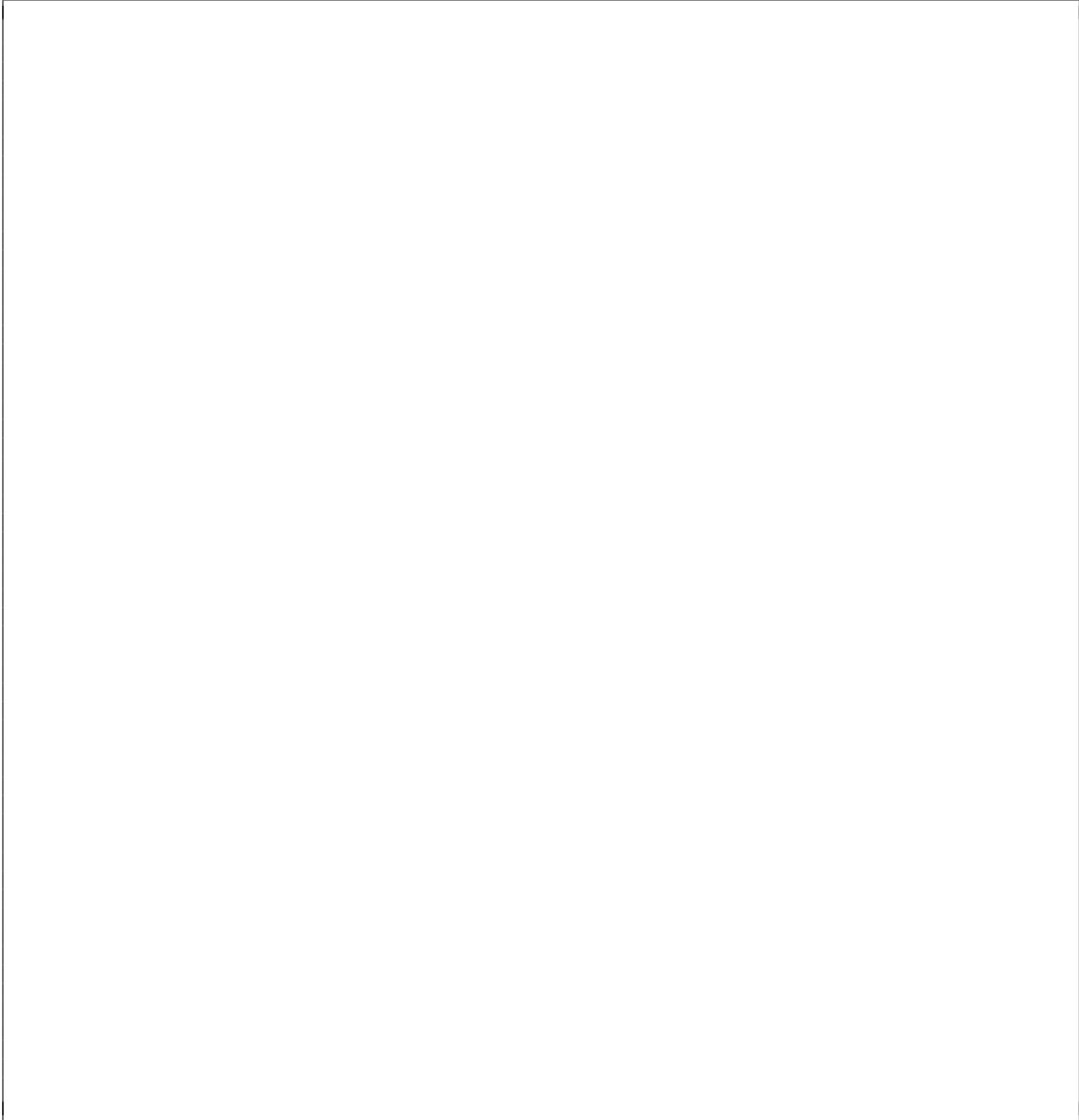


Question 9: FPGAs and Verilog [15 marks]

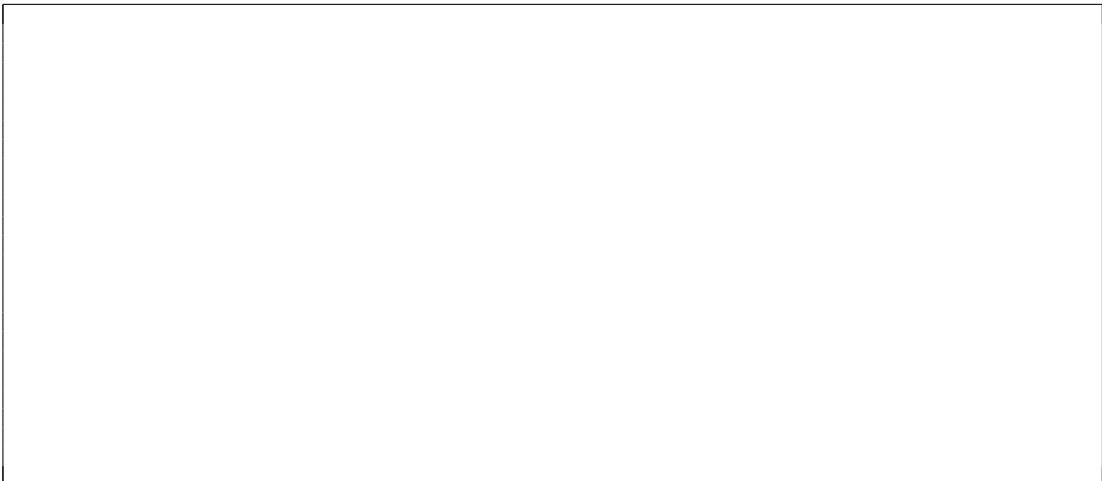
- a. FPGAs are considered flexible and highly parallel programmable logic chips that are widely used in prototypes and final versions of embedded systems. List and briefly describe any four of the seven consecutive steps of the FPGA development flow.

(8)

- b. Verilog is all about modules, ports, wires, and registers. Write a simple 2-input AND gate as a module to show the use of these ingredients; the module name should be your student number. (5)



- c. Modules are built from other modules and/or built-in modules; these other modules are instantiated in the module you are implementing. Why do we give instances names? (2)



Question 10: Verilog Simulation, CPU Architecture and Datapath [16 marks]

a. A 4-bit up-counter is implemented in the Verilog pseudocode shown below:

(9)

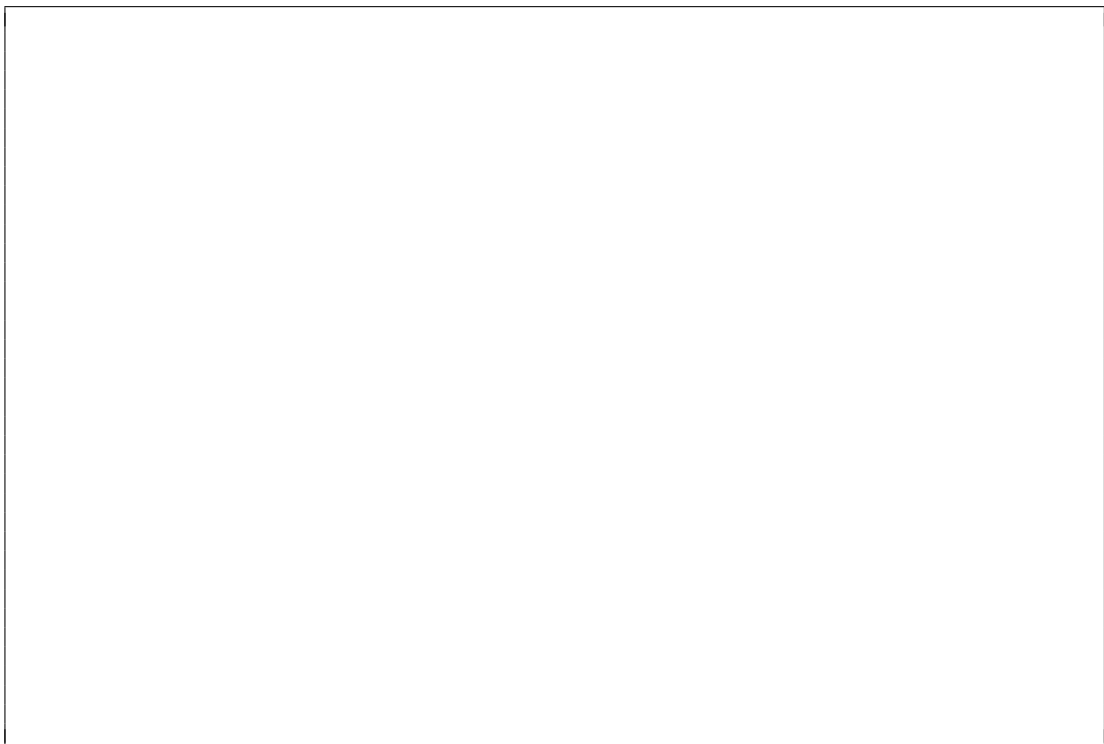
```
module counter (clk, reset, enable, count); // Define port types and directions
    input clk, reset, enable;
    output [3:0] count;
    reg [3:0] count;
    always @ (posedge clk)
        if (reset == 1'b1) begin
            count <= 0;
        end else if ( enable == 1'b1) begin
            count <= count + 1;
        end
endmodule
```

To test the functionality of the module and observe its behavior during simulation, write a testbench that initialises the input signals, toggles the clock signal, and monitors the values of **clk**, **reset**, **enable**, and **count** at different simulation time steps.



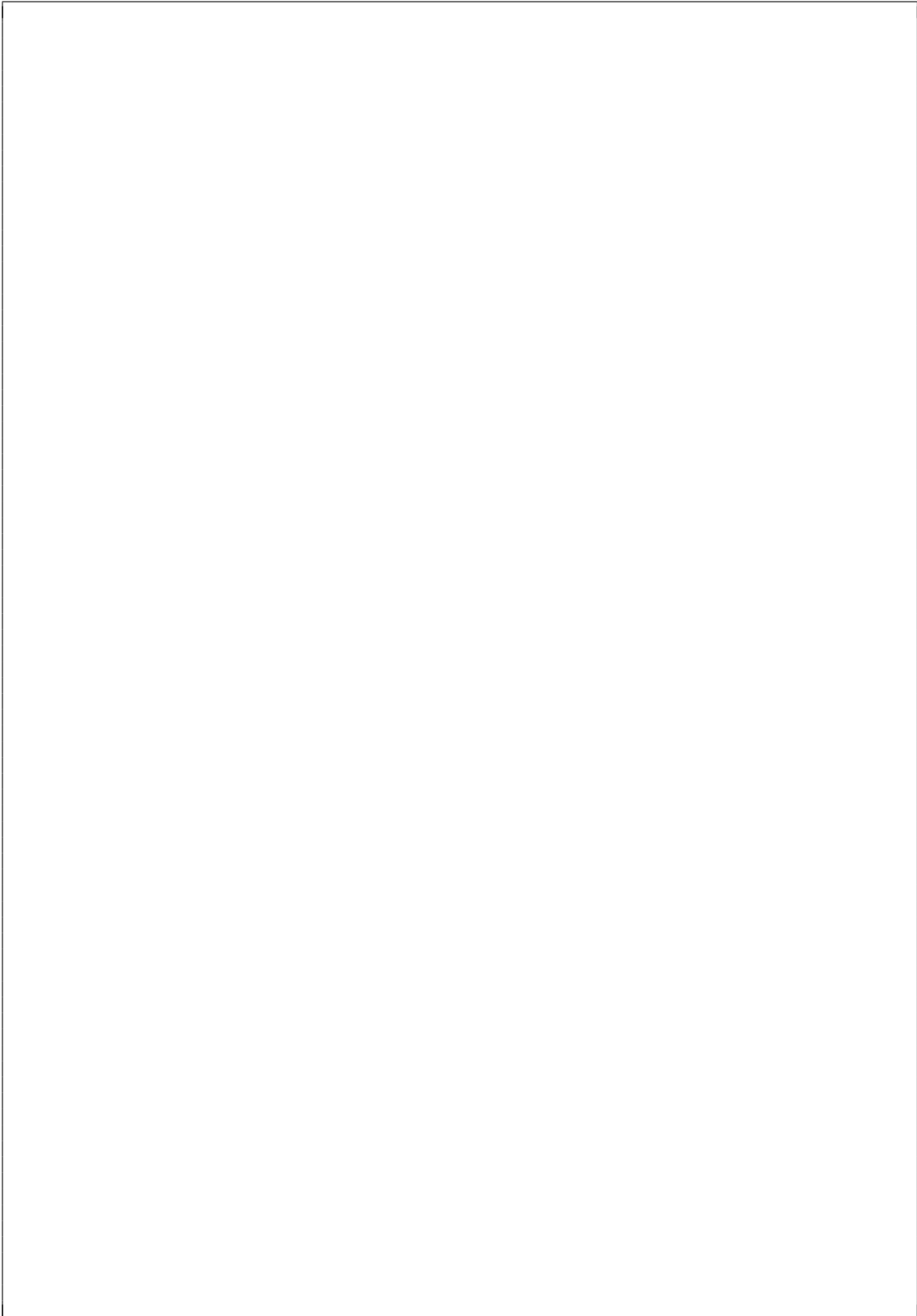
- b. The Function Unit (FU) carries out the computer operations and typically contains the Arithmetic Logic Unit (ALU) and the Control Unit (CU), i.e., $FU = ALU + CU$. What are the two main functions of the Control Unit?

(2)



- c. With the aid of a diagram, explain the operation of the FU block in a single-cycle computer.

(5)



END OF EXAM

Appendix A: Verilog Cheat Sheet

Numbers and constants

Example: 4-bit constant 10 in binary, hex and in decimal: 4'b1010 == 4'ha -- 4'd10

(numbers are unsigned by default)

Concatenation of bits using {}

4'b1011 == {2'b10, 2'b11}

Constants are declared using parameter:

parameter myparam = 51

Operators

Arithmetic: and (+), subtract (-), multiply (*), divide (/) and modulus (%) all provided.

Shift: left (<<), shift right (>>)

Relational ops: equal (==), not-equal (!=), less-than (<), less-than or equal (<=), greater-than (>), greater-than or equal (>=).

Bitwise ops: and (&), or (|), xor (^), not (~)

Logical operators: and (&&) or (||) not (!) note that these work as in C, e.g. (2 && 1) == 1

Bit reduction operators: [n] n=bit to extract

Conditional operator: ? to multiplex result

Example: (a==1)? funcif1 : funcif0

The above is equivalent to:

((a==1) && funcif1)

|| ((a!=1) && funcif0)

Registers and wires

Declaring a 4 bit wire with index starting at 0:

wire [3:0] w;

Declaring an 8 bit register:

reg [7:0] r;

Declaring a 32 element memory 8 bits wide:

reg [7:0] mem [0:31]

Bit extract example:

r[5:2] returns 4 bits between pos 2 to 5 inclusive

Assignment

Assignment to wires uses the assign primitive outside an always block, e.g.:

assign mywire = a & b

Registers are assigned to inside an always block which specifies where the clock comes from, e.g.:

always@(posedge myclock)

cnt = cnt + 1;

Blocking vs. unblocking assignment <= vs. =

The <= assignment operator is non-blocking (i.e. if use in an always@(posedge) it will be performed on every positive edge. If you have many non-blocking assignments they will all be updated in parallel. The <= operator must be used inside an always block – you can't use it in an assign statement.

The blocking assignment operator = can be used in either an assign block or an always block. But it causes assignments to be performed in sequential order. This tends to result in slower circuits, so avoid using it (especially for synthesized circuits) unless you have to.

Case and if statements

Case and if statements are used inside an always block to conditionally update state. e.g.:

```
always @(posedge clock)
  if (add1 && add2) r <= r+3;
  else if (add2) r <= r+2;
  else if (add1) r <= r+1;
```

Note that we don't need to specify what happens when add1 and add2 are both false since the default behavior is that r will not be updated. Equivalent function using a case statement:

```
always @(posedge clock)
  case({add2,add1})
    2'b11 : r <= r+3;
    2'b10 : r <= r+2;
    2'b01 : r <= r+1;
    default: r <= r;
  endcase
```

Module declarations

Modules pass inputs, outputs as wires by default.

```
module ModName (
  output reg [3:0] result, // register output
  input [1:0] bitsin, input clk, inout bidirectnl );
  ... code ...
endmodule
```

Verilog Simulation / ISIM commands

```
$display ("a string to display");
$monitor ("like printf. Vals: %d %b", decv,bitv);
#100 // wait 100ns or simulation moments
$finish // end simulation
```

Appendix B: ARM Assembly Language Cheat Sheet

Memory access instructions

```
LDR Rd, [Rn]           ; load 32-bit number at [Rn] to Rd
LDR Rd, [Rn,#off]      ; load 32-bit number at [Rn+off] to Rd
STR Rt, [Rn]           ; store 32-bit Rt to [Rn]
STR Rt, [Rn,#off]      ; store 32-bit Rt to [Rn+off]
PUSH {Rt}              ; push 32-bit Rt onto stack
POP {Rd}               ; pop 32-bit number from stack into Rd

MOV{S} Rd, <op2>       ; set Rd equal to op2
MOV Rd, #iml6           ; set Rd equal to iml6, iml6 is 0 to 65535
MVN{S} Rd, <op2>       ; set Rd equal to -op2
```

Branch instructions

```
B label                ; branch to label Always
BEQ label              ; branch if Z == 1 Equal
BNE label              ; branch if Z == 0 Not equal
BCS label              ; branch if C == 1 Higher or same, unsigned ≥
BHS label              ; branch if C == 1 Higher or same, unsigned ≥
BCC label              ; branch if C == 0 Lower, unsigned <
BLO label              ; branch if C == 0 Lower, unsigned <
BMI label              ; branch if N == 1 Negative
BPL label              ; branch if N == 0 Positive or zero
BVS label              ; branch if V == 1 Overflow
BVC label              ; branch if V == 0 No overflow
BHI label              ; branch if C==1 and Z==0 Higher, unsigned >
BLS label              ; branch if C==0 or Z==1 Lower or same, unsigned ≤
BGE label              ; branch if N == V Greater than or equal, signed ≥
BLT label              ; branch if N != V Less than, signed <
BGT label              ; branch if Z==0 and N==V Greater than, signed >
BLE label              ; branch if Z==1 or N!=V Less than or equal, signed ≤
BX Rm                  ; branch indirect to location specified by Rm
BL label               ; branch to subroutine at label
BLX Rm                 ; branch to subroutine indirect specified by Rm
```

Logical instructions

```
AND{S} {Rd}, Rn, <op2> ; Rd=Rn&op2 (op2 is 32 bits)
ORR{S} {Rd}, Rn, <op2> ; Rd=Rn|op2 (op2 is 32 bits)
EOR{S} {Rd}, Rn, <op2> ; Rd=Rn^op2 (op2 is 32 bits)
BIC{S} {Rd}, Rn, <op2> ; Rd=Rn&(~op2) (op2 is 32 bits)
ORN{S} {Rd}, Rn, <op2> ; Rd=Rn|(~op2) (op2 is 32 bits)
LSR{S} Rd, Rm, Rs      ; logical shift right Rd=Rm>>Rs (unsigned)
LSR{S} Rd, Rm, #n      ; logical shift right Rd=Rm>>n (unsigned)

ASR{S} Rd, Rm, Rs      ; arithmetic shift right Rd=Rm>>Rs (signed)
ASR{S} Rd, Rm, #n      ; arithmetic shift right Rd=Rm>>n (signed)
LSL{S} Rd, Rm, Rs      ; shift left Rd=Rm<<Rs (signed, unsigned)
LSL{S} Rd, Rm, #n      ; shift left Rd=Rm<<n (signed, unsigned)
```

Arithmetic instructions

```
ADD{S} {Rd}, Rn, <op2> ; Rd = Rn + op2
ADD{S} {Rd}, Rn, #iml2 ; Rd = Rn + iml2, iml2 is 0 to 4095
SUB{S} {Rd}, Rn, <op2> ; Rd = Rn - op2
SUB{S} {Rd}, Rn, #iml2 ; Rd = Rn - iml2, iml2 is 0 to 4095
RSB{S} {Rd}, Rn, <op2> ; Rd = op2 - Rn
RSB{S} {Rd}, Rn, #iml2 ; Rd = iml2 - Rn
CMP Rn, <op2>          ; Rn - op2 sets the NZVC bits
MUL{S} {Rd}, Rn, Rm     ; Rd = Rn * Rm signed or unsigned
```

Notes

Ra Rd Rm Rn Rt represent 32-bit registers

value any 32-bit value: signed, unsigned, or address

{S} if S is present, instruction will set condition codes

#im12 any value from 0 to 4095

#im16 any value from 0 to 65535

{Rd,} if Rd is present Rd is destination, otherwise Rn

#n any value from 0 to 31

#off any value from -255 to 4095

label any address within the ROM of the microcontroller

op2 the value generated by <op2>